MOVIE SUCCESS PREDICTOR

Overall Project Report

Data Science and Visualization (24ADI204) | October 2026

Team: Sunil E, Ragunath T, Dejashwin R P, Vishwaanth N

1. Introduction and Problem Statement

Predicting the success of a movie is challenging because its performance depends on several factors, such as production budget, genre, cast, director, runtime, and release timing. Producers and investors often struggle to estimate a movie's potential performance before its release.

The Movie Success Predictor uses historical movie data and machine learning to predict whether a movie will be successful, so that producers can judge financial risk before release and make more informed decisions. In this project, a movie is labelled successful when its IMDb score is 7.0 or higher.

2. Objectives

1. Analyze historical movie data and identify factors influencing movie success.
2. Examine the relationship between budget, genre, cast, director, runtime and release timing.
3. Develop a machine learning model to predict whether a movie will be successful.
4. Evaluate the model's performance using appropriate evaluation metrics.
5. Visualize movie trends and prediction results through charts and graphs.
6. Help producers judge financial risk before release by estimating a movie's chance of success.

3. Scope

In scope: data cleaning and quality checks; exploratory data analysis (statistics, charts, correlations); real release months from the TMDB API; release_season feature engineering; a movie success prediction model; a pre-release risk view for producers.

Out of scope: live or real-time box-office tracking; proving that one factor causes success; guaranteed forecasts for future films; predicting the exact gross revenue amount.

4. Dataset Description

The project uses the IMDb 5000 Movie Dataset from Kaggle.

Attribute	Value
Source	Kaggle – IMDb 5000 Movie Dataset
Records	4,998 movies
Columns	28 (16 numeric, 12 text)
Release years	1916 – 2016 (median 2005)
Target	success (1 = IMDb score 7.0 or higher, 0 = otherwise)

Data type	Structured table (CSV)

Key fields include movie_title, director_name, genres, country, language, title_year, duration, budget, gross, imdb_score, num_voted_users, review counts and Facebook-likes counts.

5. Methodology

The workflow has six steps:

1. Collect – load the IMDb 5000 CSV from Kaggle.
2. Clean – check missing values, duplicates, zeros and outliers.
3. Explore (EDA) – statistics, charts and correlations.
4. Feature engineering – replace the dummy release_month with real months from TMDB, then derive release_season.
5. Model – train classification models to predict success.
6. Evaluate – compare models using ROC AUC, accuracy and F1.

6. Data Preprocessing Results

- There are no missing cells and no fully duplicate rows.
- Many zeros appear in social-media style columns, which most likely mean “not recorded” rather than a true zero: movie_facebook_likes 43.3%, facenumber_in_poster 42.7%, director_facebook_likes 17.9%, actor_3_facebook_likes 1.8%, actor_2_facebook_likes 1.1%.
- Budget has extreme values (minimum 218, maximum 12.2 billion), which points to currency or unit errors.
- Some titles repeat (King Kong, Home and Ben-Hur appear three times each), so movies must be matched by title and year together.
- Gross and budget are strongly right-skewed, so log transformation is recommended.

7. Exploratory Data Analysis Results

Variable	Mean	Median
imdb_score	6.44	6.6
duration (min)	107.2	103
gross (USD)	44.3 M	25.5 M
budget	37.8 M	20.0 M

- Average gross is far above the median because a few blockbusters pull it up.
- IMDb scores are roughly bell-shaped, peaking between 6.5 and 7.5.
- Most films run 90 to 120 minutes, and the data is weighted toward recent films.
- Gross is most associated with num_voted_users and num_user_for_reviews (correlation about 0.5 to 0.6). Votes and IMDb score correlate at about 0.41.
- Budget looks almost uncorrelated with everything, mainly because of its extreme outliers

rather than evidence that spending does not matter.

- Drama is the most common genre, followed by Comedy, Thriller, Action and Romance.

8. Feature Engineering: Real Release Month and Release Season

Why the release_month column must be replaced. In the feature-engineered dataset, release_month is almost evenly spread (387 to 461 movies in every month) and shows no meaningful link with success (success rate 31.0% to 40.6% across months, chi-square p = 0.17). A Random Forest scored AUC 0.884 with the engineered columns and 0.887 without them. These values behave like randomly generated numbers, so they cannot be trusted.

How real months are obtained. A Python script for Google Colab reads the project CSV, matches each movie in the TMDB database using movie_title and title_year, and converts the release date to a month number (January = 1 to December = 12). It prefers the US theatrical release date. The documented fallback order is US limited theatrical, then US premiere, then TMDB's primary release date, and the source used is recorded for every row.

Safety rules built into the script:

- Every original row, including duplicates, is preserved.
- Existing values are never changed; only blank release_month cells are filled.
- Months are never guessed. Movies that are ambiguous, unmatched or without a release date stay blank and are listed in a separate file for manual verification.
- Lookups are cached, and API errors and rate limits are handled.

Current status. The script is written and tested against a simulated TMDB service, but it has not yet been run on the real dataset, so no real-month results are reported here.

Release season. After the real months are collected, release_season will be derived as follows:

Season	Months
Winter	December, January, February
Spring	March, April, May
Summer	June, July, August
Autumn	September, October, November

9. Baseline Modelling Results

Baseline classifiers were trained to predict the success label using stratified 5-fold cross-validation, default settings and a 0.5 threshold. imdb_score was excluded because it defines the label and would cause leakage (a meaningless perfect AUC of 1.000).

Feature-engineered dataset (4,998 movies):

Model	ROC AUC	Accuracy	F1 (successful)
Majority-class baseline	0.500	0.648	–
Logistic Regression	0.813	0.770	0.608
Gradient Boosting	0.876	0.806	0.688
Random Forest	0.884	0.816	0.711

Random Forest was the best honest model, about 16.8 points above the baseline accuracy. The most important features were num_voted_users, duration and num_user_for_reviews.

PCA dataset (3,998 movies, 23 components): Random Forest reached AUC 0.826 and accuracy 77.4%, Gradient Boosting AUC 0.817 and Logistic Regression AUC 0.807. Success-related signal is spread across many small components, so all 23 should be kept.

The classes are uneven: 35.2% of movies are successful and 64.8% are not, so a model that always predicts “not successful” is already right 64.8% of the time.

10. Key Findings

- Audience engagement (votes and reviews) is the strongest signal for both success and gross revenue.
- Tree-based models (Random Forest, Gradient Boosting) beat Logistic Regression.
- No single feature is strongly predictive on its own; apart from imdb_score, none exceeds a correlation of 0.25 with success.
- The existing release_month and marketing_budget columns show no signal and look synthetic, which is why real release months are being collected.

11. Challenges Faced

- Data quality: zeros in likes columns and impossible budget values.
- Outliers: a few blockbusters make gross and budget very skewed.
- Class imbalance: only about 35% of films reach an IMDb score of 7 or higher.
- Feature selection: votes, reviews and gross appear only after release, so a pre-release model must rely on budget, cast, director, genre, duration and release timing.
- TMDB matching: similar titles, API rate limits and missing release dates.

12. Limitations

- The dataset is observational and may contain selection bias and inconsistent financial reporting.
- Budget and gross may use different currencies or reporting conventions.
- Results from post-release features describe how reception relates to success; they do not forecast a film in advance.
- Correlation does not prove causation, and all results are estimates, not guarantees.

13. Future Work

1. Run the TMDB script, review the unverified list and derive release_season.
2. Build a pre-release feature set (budget, cast, director, genre, duration, release timing).
3. Compare Logistic Regression, Random Forest and Gradient Boosting, handling the 65/35 imbalance with class weights or threshold tuning.
4. Evaluate with ROC AUC, precision, recall and F1.

5. Present the chance of success as a risk score for producers.

14. Team Roles

Roles below are suggested and should be adjusted to match the team's actual split.

Team member	Role
Sunil E	Data collection, loading the dataset and dataset description
Ragunath T	Data cleaning: missing values, duplicates and outliers
Dejashwin R P	EDA, correlations and TMDB release-month feature engineering
Vishwaanth N	Charts, slide design and presentation
All members	Check results and prepare for the review

15. Conclusion

The IMDb 5000 dataset supports a useful first model of movie success: a tree-based classifier reaches about 0.88 AUC and 81.6% accuracy, a real but modest improvement over guessing. The strongest signals are audience votes and reviews, but these are known only after release. To give producers a genuine pre-release risk estimate, the next stage replaces the unreliable release_month with verified TMDB months, adds release_season, and rebuilds the model using only information available before launch.

16. References

1. Kaggle – IMDb 5000 Movie Dataset.
2. The Movie Database (TMDB) API – developer.themoviedb.org.
3. Project reports: EDA Report, Feature-Engineered Report, PCA Report and Movie Success Predictor Report.
4. Tools: Python, pandas, scikit-learn, matplotlib, Google Colab.
5. Course: Data Science and Visualization (24ADI204) project template.